

SISTEMA DE GESTÃO DE HOTELARIA

CONTEXTO DO PROJETO

TIPO DE EMPREENDIMENTO

- Rede de hotéis de categoria 5 estrelas, compostas por múltiplas unidades.

PÚBLICO DO SISTEMA

- O sistema será utilizado por pessoas interessadas em consultar e reservar quartos, hóspedes e funcionários responsáveis pela operação das unidades.

OBJETIVO DO PROJETO

- O projeto possui finalidade acadêmica e de portfólio, buscando demonstrar conhecimentos em engenharia de software, modelagem, arquitetura, segurança, testes e integração com sistemas externos.

Além da implementação, o projeto será utilizado para estudar e justificar escolhas técnicas realizadas durante o desenvolvimento.

ESCOPO DO SISTEMA

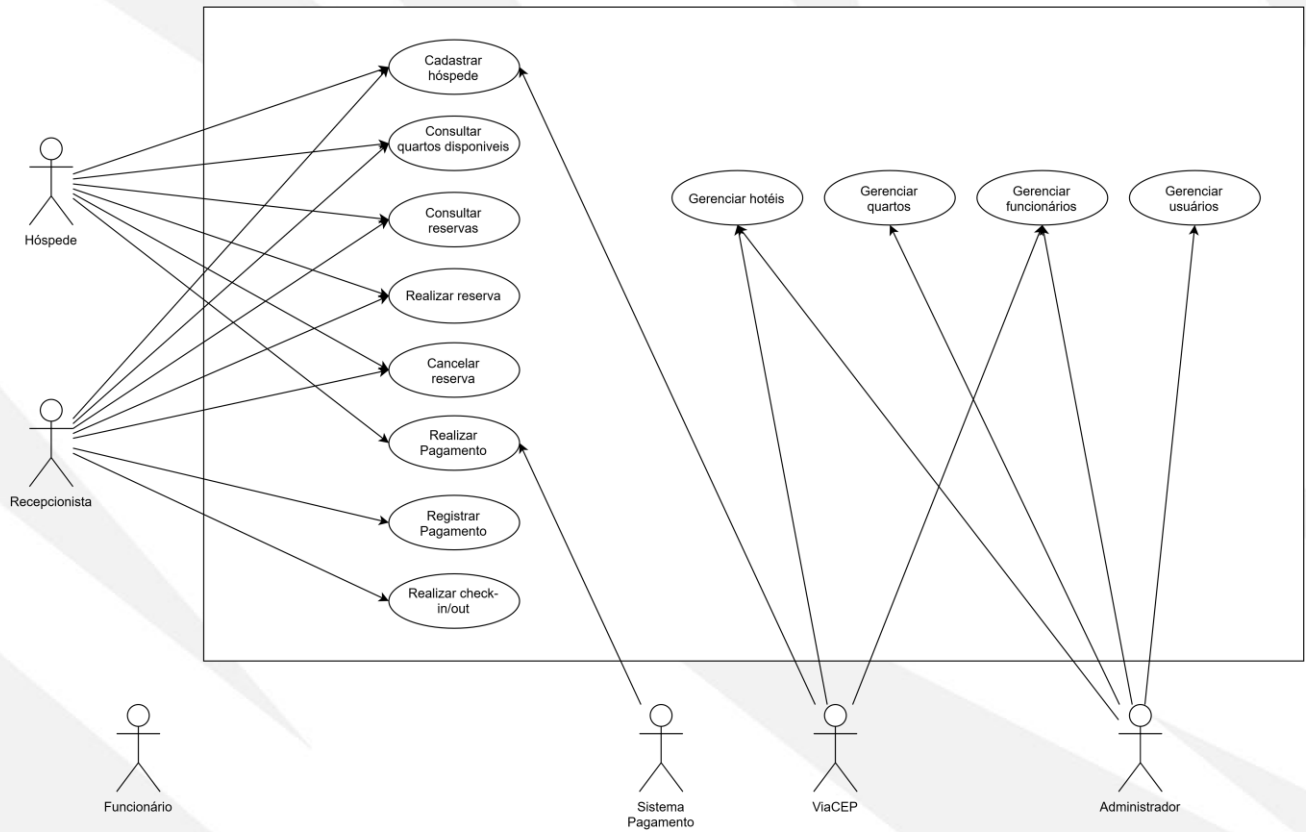
O sistema terá como escopo inicial o gerenciamento das principais operações de uma rede de hotéis 5 estrelas, contemplando:

Operações
• Gestão das unidades da rede.
• Cadastro e gerenciamento de hóspedes.
• Cadastro e gerenciamento de funcionários.
• Gestão de quartos e categorias de quartos.
• Consulta de disponibilidade.
• Criação e gerenciamento de reservas.
• Processos de check-in e check-out.
• Controle da hospedagem.
• Processamento e registro de pagamentos.
• Controle de acesso conforme o perfil do usuário.
• Consulta de endereços por meio do ViaCEP.
• Integração com serviço externo de pagamento.

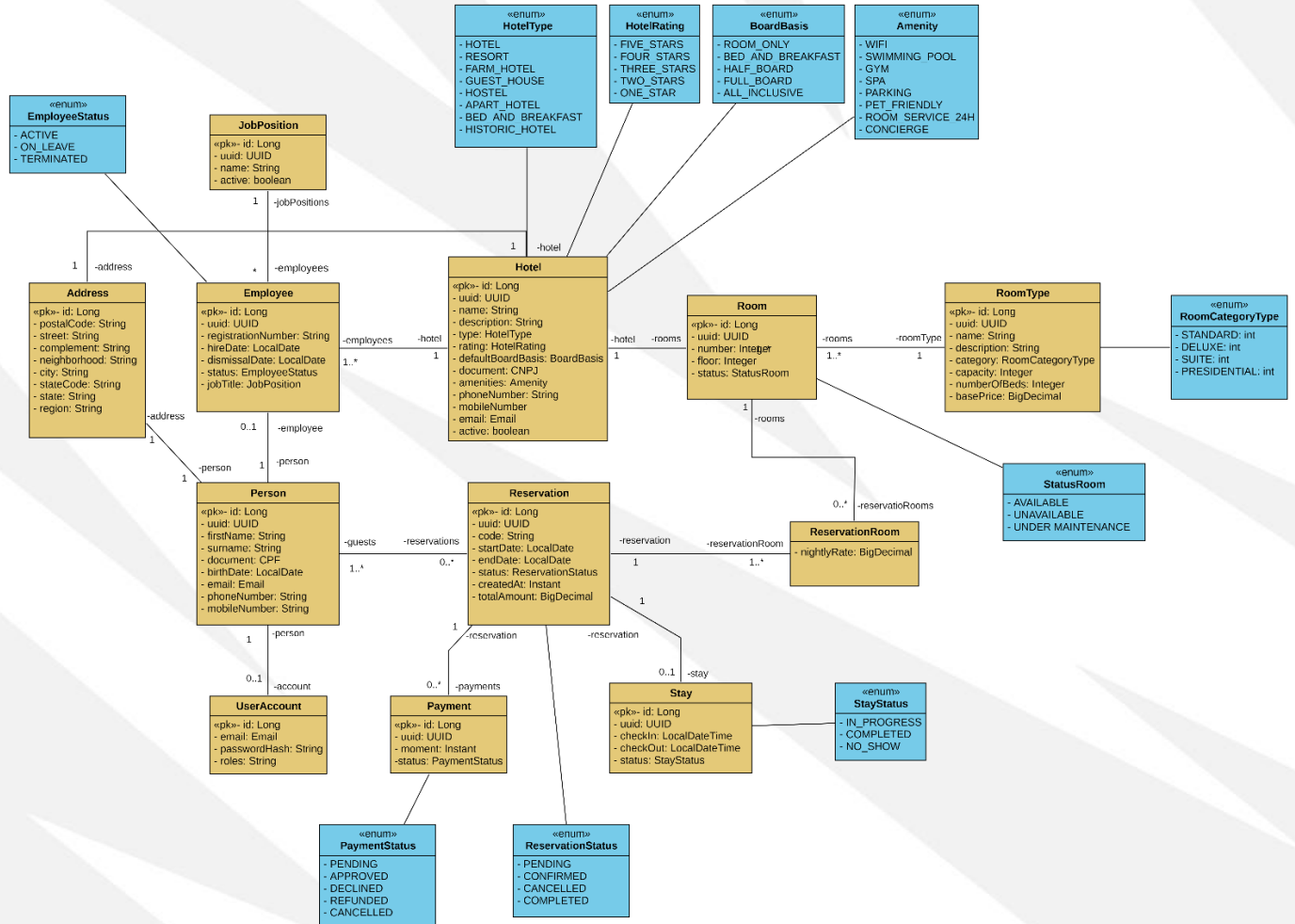
ATORES DO SISTEMA

Atores	Funções
Hóspedes	Pessoa que utiliza os serviços de hospedagem da rede.
Recepcionista	Responsável por operações relacionadas ao atendimento e à hospedagem dos clientes.
Funcionário	Colaborador da unidade hoteleira que utilizará funcionalidades relacionadas às suas atividades.
Administrador	Responsável pela administração e gerenciamento do sistema.
Sistema de Pagamento	Serviço externo utilizado para processamento de pagamentos.
ViaCEP	Serviço externo utilizado para consulta e preenchimento de informações de endereço a partir do CEP.

CASOS DE USO



MODELO CONCEITUAL



REGRAS DE NEGÓCIOS

Hotel e Quartos

- Todo **Room** deve pertencer a exatamente um **Hotel**.
- Um **Hotel** deve possuir pelo menos um quarto para operar.
- Todo **Room** deve possuir um **RoomType**.
- Um quarto indisponível ou em manutenção não pode ser reservado.
- O mesmo quarto não pode possuir reservas com períodos conflitantes.

Funcionários e acesso

- Todo **Employee** deve estar associado a uma **Person**.
- Todo **Employee** deve pertencer a um **Hotel**.
- Todo funcionário que utiliza o sistema deve possuir um **UserAccount**.
- Um usuário inativo não pode acessar funcionalidades protegidas.
- As operações disponíveis devem respeitar a **Role** do **UserAccount**.
- Um funcionário desligado não deve continuar com acesso ativo ao sistema.

Reserva

- Toda **Reservation** deve possuir pelo menos um hóspede.
- Toda **Reservation** deve possuir pelo menos um **ReservationRoom**.
- startDate** deve ser anterior a **endDate**.
- Não pode ser criada uma reserva para um período já encerrado.
- Um quarto só pode ser incluído na reserva se estiver disponível durante todo o período solicitado.
- Uma reserva cancelada não pode receber check-in.
- Uma reserva concluída não pode ser cancelada.
- Alterações no período da reserva devem validar novamente a disponibilidade dos quartos.

Hóspedes

Uma pessoa pode participar de várias reservas ao longo do tempo.

Uma reserva pode possuir vários hóspedes.

Pelo menos um hóspede deve ser definido como responsável/titular da reserva, caso decidamos manter essa informação no domínio.

A quantidade de hóspedes atribuída aos quartos não pode ultrapassar a capacidade permitida pelos respectivos **RoomType**.

Pagamento

Todo **Payment** deve pertencer a uma **Reservation**.

Uma reserva pode possuir nenhum ou vários pagamentos.

Um pagamento deve possuir valor maior que zero.

Um pagamento aprovado não pode ser aprovado novamente.

Um pagamento recusado não deve ser considerado no valor efetivamente pago.

Um estorno só pode ocorrer sobre um pagamento previamente aprovado.

A reserva só pode ser considerada totalmente paga quando a soma dos pagamentos válidos atingir o valor devido.

Stay / Hospedagem

Um **Stay** só pode ser criado para uma reserva válida.

Uma reserva pode possuir no máximo um **Stay**.

O **Stay** é criado no momento do check-in.

Não pode haver **checkout** sem **check-in**.

Um **Stay** concluído não pode voltar para **IN_PROGRESS**.

Uma reserva cancelada ou marcada como **NO_SHOW** não gera **Stay**.

ARQUITETURA

Monólito modular com princípios de clean architecture / arquitetura em camadas bem separadas.

Módulo		
Domain	Modelo e regras de negócio daquele módulo	Hotel, Room, RoomType, Address, RoomStatus, RoomCategoryType
Repository	Acesso e persistência dos dados do módulo	HotelRepository, RoomRepository, RoomTypeRepository
Service	Casos de uso e orquestração das operações	CreateHotelService, CreateRoomService, FindAvailableRoomsService
Controller	Entrada HTTP da aplicação	HotelController, RoomController, endpoints REST

DECISÕES DE ARQUITETURA

1. Arquitetura do Sistema

Decisão: Utilizar Spring Modulith com arquitetura modular por domínio.

Motivo: Manter o sistema organizado em módulos independentes, com fronteiras bem definidas, sem adicionar a complexidade operacional de uma arquitetura de microsserviços.

Alternativas consideradas: Arquitetura tradicional em camadas e microsserviços.

Consequências: Os módulos devem possuir responsabilidades bem definidas e evitar dependências indevidas entre seus componentes internos.

2. Organização por Módulos

Decisão: Organizar o projeto por funcionalidades/domínios, como user, employee, hotel, reservation e payment.

Motivo: Manter classes relacionadas ao mesmo contexto próximas e facilitar a evolução independente das funcionalidades.

Alternativas consideradas: Organização global por controller, service, repository e domain.

Consequências: Cada módulo poderá possuir suas próprias camadas internas, como domain, repository, service, controller, dto e exception.

3. Modelo de Domínio

Decisão: Priorizar um modelo de domínio rico.

Motivo: Manter regras e invariantes próximas das entidades responsáveis por elas, evitando que as entidades sejam apenas estruturas de dados.

Alternativas consideradas: Modelo anêmico com regras concentradas exclusivamente nos services.

Consequências: Entidades como Person, Reservation e Employee serão responsáveis por proteger seu próprio estado e oferecer operações específicas para modificá-lo.

4. Estratégia de Identificação

Decisão: Utilizar identificação híbrida com Long e UUID.

Motivo: Utilizar identificadores numéricos de forma eficiente internamente, sem expor diretamente chaves primárias sequenciais pela API.

Alternativas consideradas: Apenas Long ou UUID como chave primária.

Consequências:

- Long id será utilizado como PK e nas FKs.
- UUID será utilizado como identificador público dos recursos.
- Endpoints utilizarão UUID, por exemplo: /people/{uuid}.

5. Gerenciamento do Schema

Decisão: Utilizar Flyway como responsável pela criação e evolução do banco de dados.

Motivo: Manter um histórico versionado e reproduzível das alterações do schema.

Alternativas consideradas: Hibernate gerenciando automaticamente o schema através de ddl-auto.

Consequências: O Hibernate será utilizado com ddl-auto=validate, enquanto alterações estruturais serão realizadas por migrations Flyway.

6. Value Objects

Decisão: Utilizar Value Objects para conceitos que possuem regras próprias e são definidos por seus valores.

Exemplos: CPF, Email e outros conceitos semelhantes.

Motivo: Centralizar validação, normalização e comportamento dentro do próprio conceito.

Alternativas consideradas: Representar esses valores apenas como String.

Consequências: Uma instância válida de um Value Object deve sempre respeitar suas invariantes.

7. Modelagem de Person

Decisão: Utilizar Person como representação da pessoa física, independentemente do papel exercido no sistema.

Motivo: Uma mesma pessoa pode ser hóspede, acompanhante, funcionário e possuir uma conta de usuário.

Alternativas consideradas: Herança, como Employee extends Person e Guest extends Person.

Consequências: Employee e UserAccount serão associados a Person, evitando duplicação de informações pessoais.

8. Exclusão de Person

Decisão: Não permitir exclusão de Person inicialmente.

Motivo: Pessoas podem estar relacionadas a reservas, hospedagens e vínculos profissionais históricos.

Alternativas consideradas: Exclusão física e anonimização.

Consequências: A estratégia definitiva de anonimização e retenção de dados será definida posteriormente após estudo específico da LGPD.

9. Modelagem de Cargos

Decisão: Representar cargos através da entidade JobPosition.

Motivo: Evitar cargos cadastrados livremente como texto e permitir gerenciamento centralizado.

Alternativas consideradas: String livre ou enum.

Consequências: Vários funcionários poderão compartilhar o mesmo JobPosition, e novos cargos poderão ser adicionados sem alteração do código-fonte.

10. Reserva de Múltiplos Quartos

Decisão: Utilizar ReservationRoom entre Reservation e Room.

Motivo: Uma reserva pode conter diversos quartos, e a relação pode possuir informações próprias, como o valor da diária contratada.

Alternativas consideradas: Relacionamento direto muitos-para-muitos entre Reservation e Room.

Consequências: ReservationRoom passa a representar um conceito próprio dentro do domínio.

11. Reserva e Hospedagem

Decisão: Separar Reservation de Stay.

Motivo: Reservation representa aquilo que foi planejado, enquanto Stay representa a hospedagem efetivamente realizada.

Alternativas consideradas: Armazenar check-in e check-out diretamente em Reservation.

Consequências: Uma reserva pode possuir 0..1 Stay, permitindo representar reservas canceladas ou hóspedes que não compareceram.

12. Pagamentos

Decisão: Permitir vários pagamentos para uma mesma reserva.

Motivo: Uma reserva pode possuir tentativas recusadas, pagamentos parciais, múltiplas transações ou estornos.

Alternativas consideradas: Relacionamento 1:1 entre Reservation e Payment.

Consequências: Cada Payment terá seu próprio status e uma Reservation poderá possuir 0..* Payments.

13. DTOs

Decisão: Não expor diretamente as entidades de domínio através da API.

Motivo: Evitar acoplamento entre o contrato HTTP e o modelo interno da aplicação.

Alternativas consideradas: Utilizar entidades diretamente nos controllers.

Consequências: DTOs representarão entrada e saída da API, enquanto Value Objects como CPF e Email serão criados durante a conversão para o domínio.

14. Tratamento de Exceptions

Decisão: Manter exceptions específicas dentro dos respectivos módulos e utilizar uma exception base compartilhada para violações de regras de negócio.

Estrutura: BusinessException como classe base e exceptions específicas herdando dela.

Motivo: Preservar significado de domínio e reduzir repetição no tratamento HTTP.

Consequências: Um `GlobalExceptionHandler` localizado no módulo `shared` será responsável por converter `exceptions` para respostas HTTP.

15. Estratégia de Testes

Decisão: Separar testes unitários e testes de integração.

Motivo: Validar tanto regras isoladas quanto o comportamento real da aplicação.

Estratégia:

- Testes unitários executados pelo Maven Surefire.
- Testes de integração *IT executados pelo Maven Failsafe.
- RestAssured para testes dos endpoints.
- H2 como banco nos testes de integração.
- JaCoCo para cobertura consolidada.

Consequências: O relatório de cobertura considera tanto testes unitários quanto testes de integração.

16. Integrações Externas

Decisão: Isolar integrações externas do domínio principal.

Integrações previstas: ViaCEP e serviço externo de pagamento.

Motivo: Evitar que regras de domínio dependam diretamente de contratos ou tecnologias externas.

Consequências: Mudanças no provedor externo deverão ter impacto mínimo sobre o restante da aplicação.

1. Classificação por Categoria Principal (`category`)

Utiliza identificadores únicos em letras maiúsculas (`SNAKE_CASE`) para evitar problemas de tradução e acentuação no código.

- `HOTEL`: Hotel urbano ou tradicional.
- `RESORT`: Complexo de lazer de alto padrão.
- `FARM_HOTEL`: Hotel-fazenda / Turismo rural.
- `GUEST_HOUSE`: Pousadas e estadias familiares.
- `HOSTEL`: Albergues com foco em quartos compartilhados.
- `APART_HOTEL`: Flats, apart-hotéis e estadias de longa permanência.
- `BED_AND_BREAKFAST`: Cama e café.
- `HISTORIC_HOTEL`: Hotéis históricos.

2. Atributos Complementares (Filtros da API)

Para tornar a busca da API mais robusta, o tipo de hotel não deve ser a única classificação. Você deve enriquecer o objeto com **atributos booleanos** ou **arrays de propriedades**:

Perfil do Viajante (`target_audience`)

- `BUSINESS`: Focado em viagens corporativas.
- `FAMILY`: Estrutura para crianças e famílias.
- `ADULTS_ONLY`: Exclusivo para adultos / Casais.
- `BACKPACKER`: Focado em jovens e mochileiros.

Regime de Alimentação (`board_basis`)

- `ROOM_ONLY`: Sem alimentação incluída.
- `BED_AND_BREAKFAST`: Apenas café da manhã.
- `HALF_BOARD`: Meia pensão (café e almoço/jantar).
- `FULL_BOARD`: Pensão completa (3 refeições).
- `ALL_INCLUSIVE`: Tudo incluído (comidas e bebidas livres).